

NestJS

Interview Overview — Concepts and Mapping

Prepared for **Satish Kumar**

SDE-3 Node.js — WeKan Enterprise Solutions

August 2026

This is a mapping document, not a deep dive. The goal is that you can name every piece, explain the model, and be honest about depth — which is the right posture for a framework you have not shipped.

Contents

1. **Why NestJS exists**
2. **The same endpoint, three ways** — Express, NestJS, Spring Boot
3. **The complete mapping table**
4. **The four pipeline pieces** — guards, interceptors, pipes, filters
5. **Dependency injection and modules**
6. **What to say in the interview**
7. Appendix — vocabulary checklist

Scope note. You said you would manage NestJS depth yourself. This document covers what was discussed in the session: the model, the vocabulary, the Express and Spring mappings, and the honest positioning line. It does not cover Nest-specific implementation detail — custom decorators, dynamic modules, microservice transports, testing utilities — which is where you would go next if you want real depth.

1. Why NestJS exists

Express gives you a blank sheet. It handles routing and middleware and leaves everything else to you.

On a six-person team over two years, that means:

- Six different folder structures
- No dependency injection — every file `require`s its dependencies directly
- Manual wiring everywhere
- Hard-to-test code, because everything touches `req` and `res`

THE FRAMING THAT MATTERS FOR YOU

This is **exactly why Spring exists for Java**. NestJS is Express underneath with Spring Boot's structure bolted on top. Your Spring background is the asset here — you already know the architecture, you just do not know the syntax.

NestJS does not replace Express. By default it runs **on top of** Express as its HTTP adapter. Everything in the Express guide — middleware chains, the request lifecycle, `req` and `res` — is still there underneath. Nest simply gives you a structured layer above it. *(It can also run on Fastify instead, by swapping the adapter.)*

2. The same endpoint, three ways

Express

```
// routes/orders.js
router.get("/:id", auth, async (req, res, next) => {
  try {
    const order = await orderService.findById(req.params.id);
    res.json(order);
  } catch (err) { next(err); }
});
```

NestJS

```
@Controller("orders")
@UseGuards(AuthGuard)
export class OrdersController {
  constructor(private ordersService: OrdersService) {} // injected

  @Get("/:id")
  findOne(@Param("id") id: string) {
    return this.ordersService.findById(id); // just return it
  }
}
```

Spring Boot — for comparison

```
@RestController
@RequestMapping("/orders")
public class OrdersController {

  @Autowired
  private OrdersService ordersService;

  @GetMapping("/{id}")
  public Order findOne(@PathVariable String id) {
    return ordersService.findById(id);
  }
}
```

Look at what disappeared between Express and Nest: no `req`, no `res`, no try/catch, no manual service import.

NestJS handles **serialization** (whatever you return becomes the JSON response) and **error catching** (exceptions route to an exception filter) for you. **The Nest and Spring versions are structurally almost identical.**

3. The complete mapping table

Express	NestJS	Spring Boot
<code>router.get("/:id", ...)</code>	<code>@Get("/:id")</code>	<code>@GetMapping("/{id}")</code>
<code>router.post("/", ...)</code>	<code>@Post()</code>	<code>@PostMapping</code>
<code>app.use("/orders", router)</code>	<code>@Controller("orders")</code>	<code>@RequestMapping("/orders")</code>
<code>req.params.id</code>	<code>@Param("id")</code>	<code>@PathVariable</code>
<code>req.body</code>	<code>@Body()</code>	<code>@RequestBody</code>
<code>req.query</code>	<code>@Query()</code>	<code>@RequestParam</code>
<code>req.headers</code>	<code>@Headers()</code>	<code>@RequestHeader</code>
Middleware (auth)	Guard	Filter / Interceptor
Middleware (logging, transform)	Interceptor	<code>@Around</code> advice
Manual validation	Pipe	<code>@Valid</code> + Bean Validation
Error middleware (4 args)	Exception filter	<code>@ExceptionHandler</code>
<code>require("./service")</code>	Dependency injection	<code>@Autowired</code>
Folder convention	Module	<code>@Configuration</code> / component scan
<code>res.status(201)</code>	<code>@HttpCode(201)</code>	<code>@ResponseStatus</code>

READ THE MIDDLEWARE ROWS CAREFULLY — THIS IS THE REAL DIFFERENCE

Express has **one** concept, middleware, doing **four** jobs. NestJS splits it into four named things with **defined positions in the request pipeline**. That is not cosmetic — it means the framework knows what each piece is for, and can order them correctly without you thinking about registration order.

4. The four pipeline pieces

In Express you would write all four of these as generic middleware and rely on registration order. Nest gives each a name and a fixed slot.

Piece	Job	Express equivalent
Guard	Decide whether the request may proceed — auth, roles, permissions. Returns true or false.	An <code>auth</code> middleware that either calls <code>next()</code> or sends 401
Interceptor	Wrap the handler — runs before <i>and</i> after. Logging, timing, response transformation, caching.	Middleware that also needs to act on the way out (awkward in Express)
Pipe	Transform and validate the incoming data before it reaches the handler.	A <code>zod</code> or <code>joi</code> validation middleware
Exception filter	Catch anything thrown and turn it into an HTTP response.	The four-argument error middleware

The interceptor is the one with no clean Express equivalent. Express middleware runs *before* the handler; getting code to run reliably *after* it means monkey-patching `res.json` or listening on `res.on("finish")`. A Nest interceptor wraps the handler on both sides natively — which is why response shaping and timing live there.

The pipeline order

```
Request
-> Middleware      (still supported, Express-style)
-> Guard           (may reject: 403)
-> Interceptor     (before part)
-> Pipe            (validate and transform @Body, @Param)
-> HANDLER
-> Interceptor     (after part)
-> Exception filter (only if something threw)
Response
```

Note that Nest still supports plain Express middleware for the cases where you want it — helmet, cors, compression are all applied as middleware in a Nest app exactly as they would be in Express.

NestJS never had the async error problem

WORTH SAYING
Express 4's async handlers need a `.catch(next)` wrapper, because Express ignores a handler's return value and an async rejection escapes. **NestJS never had this problem** — exception filters handle async errors out of the box. If the Express 4 async trap comes up, mentioning that Nest solves it natively is a good connection to draw.

5. Dependency injection and modules

Dependency injection

```
@Injectable()
export class OrdersService {
  constructor(private ordersRepo: OrdersRepository) {}
}

@Controller("orders")
export class OrdersController {
  constructor(private ordersService: OrdersService) {} // Nest provides it
}
```

You **declare what you need** in the constructor; Nest constructs and provides it. You never call `new OrdersService()` and you never `require` it.

Why this is the biggest win: testing becomes trivial. Inject a mock service instead of a real one and the controller has no idea. In Express, a hardcoded `require("./orderService")` means you have to mock the module system itself.

This is identical to Spring's `@Autowired`. Same container, same lifecycle, same singleton-by-default behaviour.

Modules

```
@Module({
  imports:      [DatabaseModule],      // what this module needs
  controllers:  [OrdersController],    // its HTTP entry points
  providers:    [OrdersService],       // its injectable services
  exports:      [OrdersService]        // what other modules may use
})
export class OrdersModule {}
```

Code is grouped into `OrdersModule`, `UsersModule` and so on, each declaring explicitly what it needs and what it exposes.

THE POINT

Structure is enforced, not conventional. In Express, a service in one folder can be imported by anything, anywhere — boundaries exist only by agreement. In Nest, if `OrdersService` is not in `exports`, another module simply cannot inject it. The architecture is checked by the framework.

The third thing: TypeScript-first

- Types on request bodies via DTO classes
- Automatic validation through pipes (`class-validator` decorators on the DTO)
- Generated OpenAPI / Swagger docs from those same types

6. What to say in the interview

Do not fake depth on a framework you have not shipped. One follow-up question — "how do you write a custom pipe?", "how does a dynamic module work?" — and it shows. Honest positioning plus demonstrated understanding of the model is a much stronger position than a thin claim of experience.

THE LINE TO USE

"I have built primarily in Express, so I know the middleware pipeline well — and my Spring Boot background maps closely onto NestJS. Nest is Express underneath with a structured layer on top: controllers with decorators instead of route handlers, dependency injection instead of manual imports, and guards, interceptors, pipes and exception filters replacing what Express does with generic middleware. The DI and module model is essentially Spring's, which is why I would be productive quickly."

Why this works. You are not claiming NestJS depth. You are demonstrating that you:

- Know what Nest *is* architecturally, not just that it exists
- Can name every piece of the pipeline correctly
- Have genuinely relevant transferable experience (Spring DI, modules, layered architecture)
- Are honest about your own gaps — which senior interviewers weight heavily

Likely follow-ups and short answers

Question	Answer shape
"What is a guard?"	Decides whether a request may proceed. Returns a boolean. Auth and role checks live here. Equivalent to an auth middleware in Express.
"Guard vs interceptor?"	A guard decides <i>if</i> the handler runs. An interceptor wraps the handler — it runs before and after, so it is for logging, timing and response shaping.
"Why use Nest over Express?"	Structure and testability on a larger team. DI makes services mockable, modules enforce boundaries, and the pipeline pieces have defined roles rather than being ordered middleware.
"Does Nest replace Express?"	No — it runs on top of Express by default as its HTTP adapter, and can swap to Fastify.
"How would you handle validation?"	A DTO class with class-validator decorators, applied by a validation pipe. Equivalent to <code>@Valid</code> in Spring.

Appendix — vocabulary checklist

You should be able to define each of these in one sentence. That is the bar for this interview.

Term	One-sentence definition
Controller	A class whose methods are HTTP route handlers, marked with <code>@Controller</code> and method decorators.
Provider / Service	An injectable class holding business logic, marked <code>@Injectable</code> .
Module	A grouping unit declaring its controllers, providers, imports and exports.
Guard	Runs before the handler and returns true or false to allow or reject the request.
Interceptor	Wraps the handler on both sides — logging, timing, response transformation, caching.
Pipe	Validates and transforms incoming arguments before they reach the handler.
Exception filter	Catches thrown exceptions and converts them into HTTP responses.
DTO	Data Transfer Object — a class describing the shape of a request body, used for typing and validation.
Decorator	The <code>@Something</code> syntax that attaches metadata to a class, method or parameter.
DI container	The registry that constructs providers and injects them where declared — same concept as Spring's application context.

If you want real depth later, the order to learn it in: modules and DI first (you already know the concept from Spring), then the four pipeline pieces with one custom implementation of each, then DTOs with class-validator, then testing with Nest's `Test.createTestingModule`. That last one is where the DI payoff becomes obvious, and it is what an interviewer means by "have you actually used it."